Spring MVC

Од HTTP барање до HTML страница и JSON одговор

1 — 2

Час 1 Час 2

Универзитет „Св. Кирил и Методиј“ во Скопје
ФАКУЛТЕТ ЗА ИНФОРМАТИЧКИ НАУКИ
И КОМПЈУТЕРСКО ИНЖЕНЕРСТВО

Цели

Потсетник

- HTTP: барање, одговор, методи, статус кодови
- Сервлет: `doGet`, `doPost`
- Бин, IoC, вметнување зависности
- `@Service`, `@Repository`
- Апликација за известувања

Денес ќе научиме

1. **MVC шаблон** и **слоевита архитектура**
2. Што прави **DispatcherServlet**
3. Патот на едно барање низ Spring MVC
4. Прв контролер: `@Controller` и преглед (HTML)
5. `@Controller` или `@RestController`?
6. Мапирање: `@RequestMapping`, `@GetMapping`…

Каде застанавме: сервлет

```java
@WebServlet("/notifications")
public class NotificationServlet extends HttpServlet {
    private NotificationService service = ...;   // кој го креира?

    protected void doGet(HttpServletRequest req,
                         HttpServletResponse resp) throws IOException {
        String to = req.getParameter("to");                // рачно читање
        String limitParam = req.getParameter("limit");
        int limit = (limitParam == null) ? 10
                  : Integer.parseInt(limitParam);          // рачна конверзија

        List<Notification> list = service.findByTo(to, limit);

        resp.setContentType("text/html;charset=UTF-8");
        PrintWriter out = resp.getWriter();
        out.println("<ul>");                                // HTML во Java
        for (Notification n : list)
            out.println("<li>" + n.message() + "</li>");
        out.println("</ul>");
    }
}
```

Проблеми

Рачно ги читаме параметрите (сè е `String`)

Рачно конвертираме `String` → `int`

HTML е измешан со Java кодот

Нема DI — кој го креира сервисот?

Ист код се повторува во секој сервлет

Истото со Spring MVC

```java
@Controller
public class NotificationController {
    private final NotificationService service;

    public NotificationController(NotificationService service) {
        this.service = service;                          // DI
    }

    @GetMapping("/notifications")                        // URL → метод
    public String list(@RequestParam String to,
                       @RequestParam(defaultValue = "10") int limit,
                       Model model) {
        model.addAttribute("notifications",
                           service.findByTo(to, limit));
        return "notification-list";   // име на HTML шаблоном
    }
}
```

Решено

✓ `@GetMapping` — URL → метод

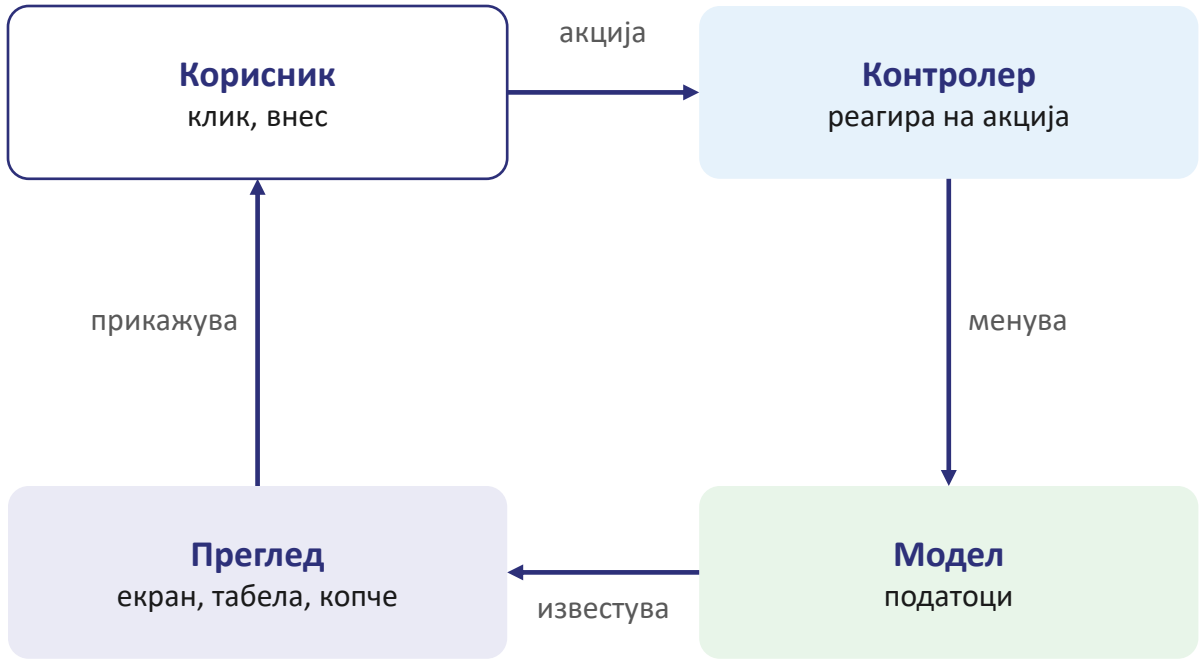
✓ `@RequestParam int` — Spring чита и конвертира

✓ HTML е во посебна датотека (преглед)

✓ Сервисот е вметнат преку конструктор

Прашањето за денес: **кој** ги прави сите тие работи наместо нас? → **Spring MVC**

MVC шаблон: Модел – Преглед – Контролер



Модел

Податоците што се прикажуваат

Преглед (View)

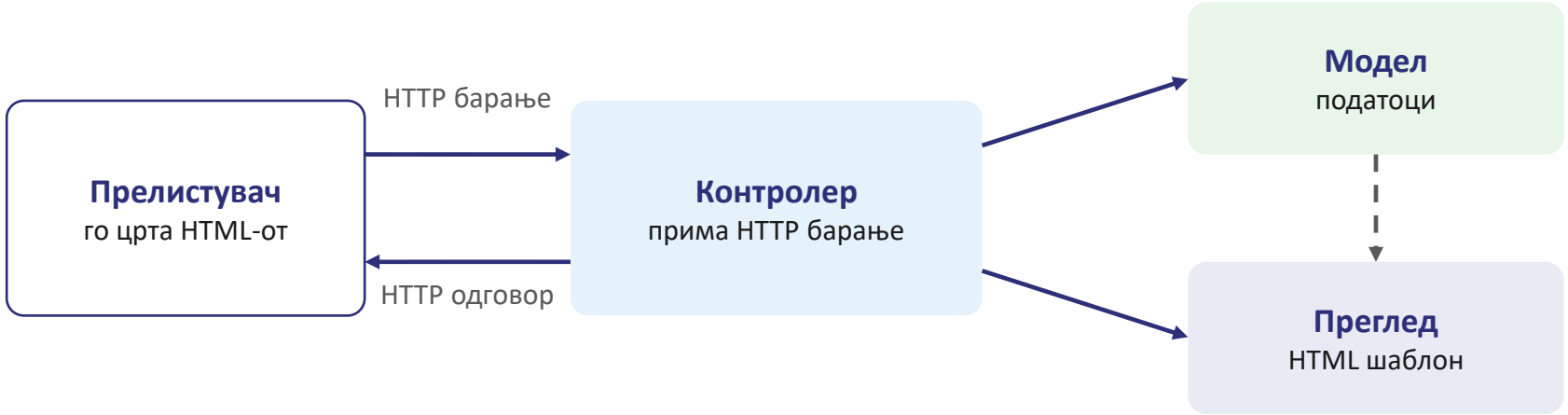
Визуелните елементи: табели, полиња, копчиња. Зависи од моделот

Контролер

Се справува со акциите на корисникот, го менува моделот, може да го смени прегледот

Циклус: **акција → промена на моделот → повторно исцртување на прегледот**

MVC шаблон за веб апликации



Прегледот се пополнува со податоците од моделот и станува **HTML** во одговорот

- Корисникот комуницира **директно со контролерот** преку HTTP
- Секоја акција (клик, форма) = **ново HTTP барање**
- Прегледот најчесто е **шаблон** (template) што генерира HTML

Улогата на **моделот** е иста; улогите на контролерот и прегледот се прилагодени на HTTP.

Контролер - што сè прави?

```java
@GetMapping("/notify")
public String notifyUser(@RequestParam String to,
                         @RequestParam String msg, Model model) {
    // 1. податоци од барањето: to, msg

    // 2. бизнис правило
    if (msg.length() > 160)
        throw new IllegalArgumentException("Предолга порака");

    // 3. праќање
    new SmtpClient("smtp.finki.ukim.mk").send(to, msg);

    // 4. зачувување
    jdbc.update("INSERT INTO notification VALUES (?, ?)", to, msg);

    // 5. модел и избор на преглед
    model.addAttribute("to", to);
    return "sent";
}
```

Што не е во ред?

S — една класа, пет одговорности

O — SMS наместо e-mail? База наместо листа? Секогаш менуваме **контролер**

Тест — не може да се тестира без SMTP и база

Решение: **слоеви** — секој слој една одговорност

Слоевита архитектура

Преглед (UI) · `notification-list.html`
како изгледа страницата

↓

Веб (презентациски) слој · `NotificationController`
HTTP ↔ Java, без бизнис логика

↓

Сервисен слој · `NotificationService`
бизнис правила (пр. макс. 160 знаци)

↓

Слој за податочен пристап · `NotificationRepository`
зачувај, прочитај, избриши

Модел
`Notification`

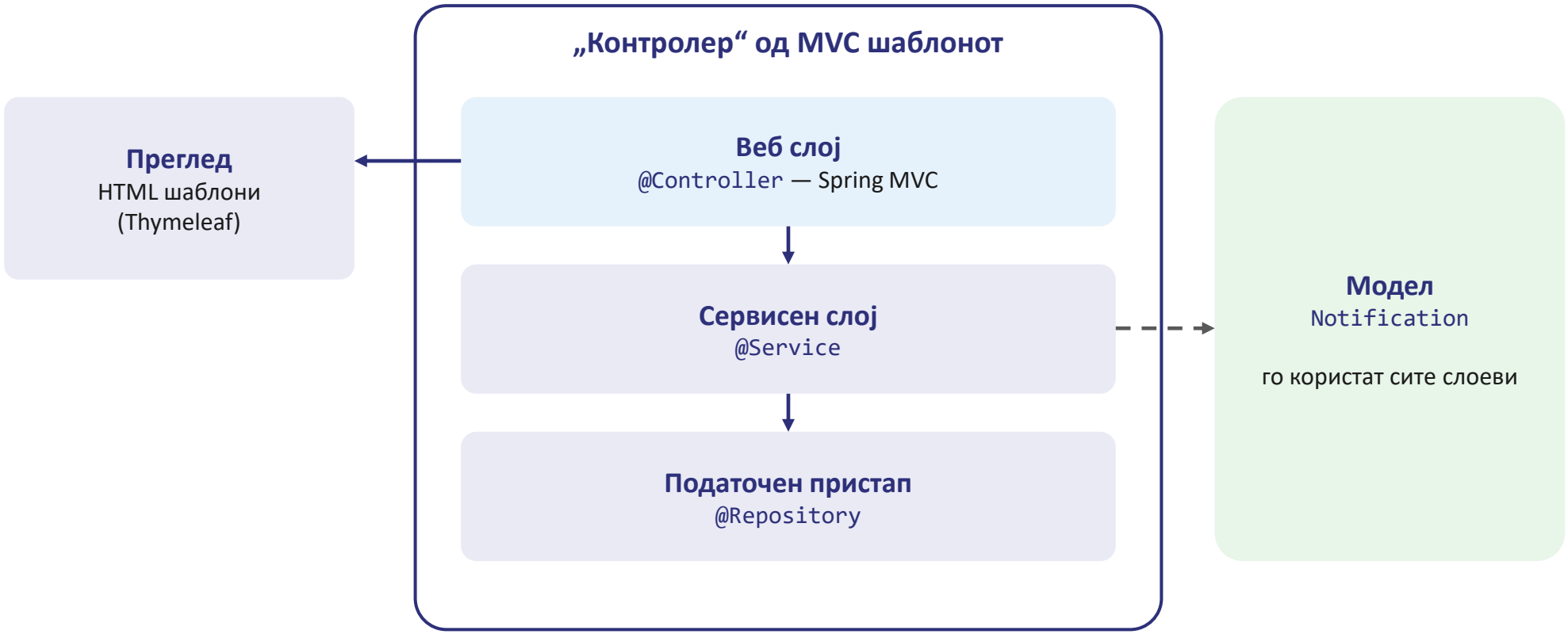
минува низ
сите слоеви

Придобивки

- Апстракција на однесувањето
- Поделба на одговорностите
- Лесна замена на компонента (листа → база)
- Полесно тестирање (лажен слој)

Зависностите одат **само надолу**. Моделот зависи само од стандардните Java класи.

Слоевит MVC шаблон



Секоја компонента има **јасно дефинирана одговорност** → ги следиме SOLID принципите.

Прашање: каде припаѓа?

1. Го чита параметарот `?to=` од URL-то
2. „Пораката не смее да биде подолга од 160 знаци“
3. Ја зачувува пораката во листа / база
4. Табела со колони „До“ и „Порака“
5. `record Notification(id, to, message, channel)`
6. Враќа статус **404** кога известувањето не постои

Избери:

Преглед

Веб слој

Сервис

Репозиториум

Модел

Одговор: каде припаѓа?

1	Го чита параметарот `?to=` од URL-то	**Веб слој**	единствен што знае за HTTP
2	„Пораката не смее да биде подолга од 160 знаци“	**Сервис**	бизнис правило
3	Ја зачувува пораката во листа / база	**Репозиториум**	податочен пристап
4	Табела со колони „До“ и „Порака“	**Преглед**	како изгледа
5	`record Notification(id, to, message, channel)`	**Модел**	податоци, без зависности
6	Враќа статус **404** кога известувањето не постои	**Веб слој**	статус кодот е дел од HTTP

Spring MVC модул

```xml
<!-- pom.xml -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>
```

spring-boot-starter-web

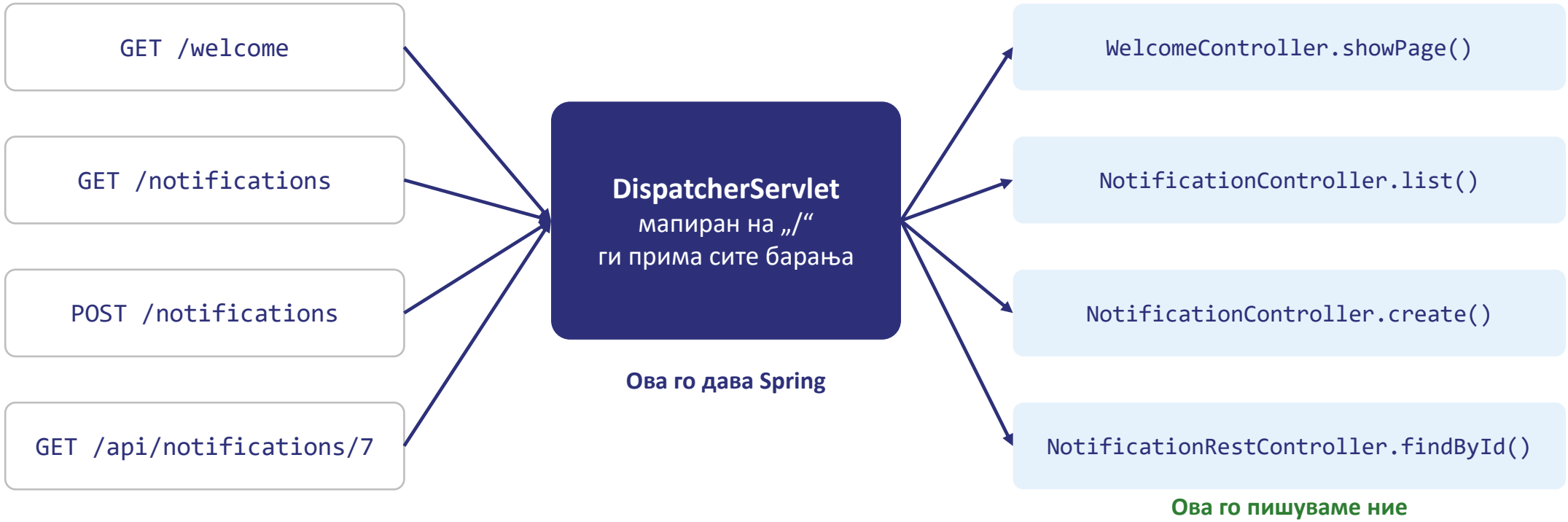
- Spring MVC (**DispatcherServlet**)
- Вграден **Tomcat** — сервлет контејнер
- **Jackson**: Java објект ↔ JSON

spring-boot-starter-thymeleaf

- HTML шаблони (прегледи)
- Готов **ViewResolver**

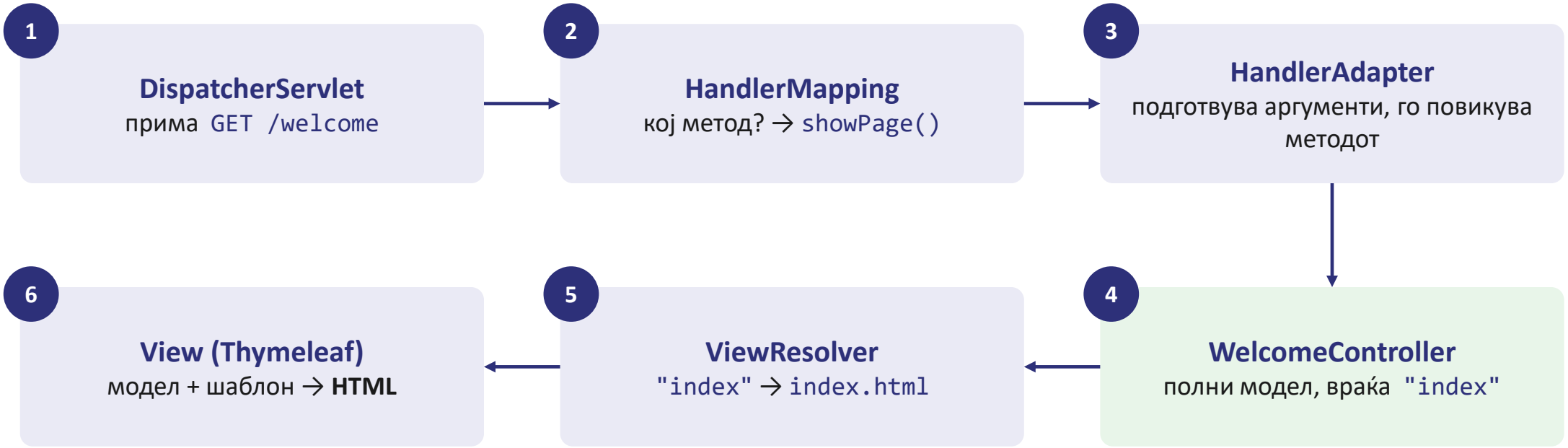
Spring MVC е изграден **врз Servlet API** — сервлети, барање, одговор, сесија, филтри и понатаму важат.

DispatcherServlet: една влезна точка



Наместо еден сервлет по URL → **еден сервлет** кој го **делегира** барањето до вистинскиот метод.

Патот на едно барање



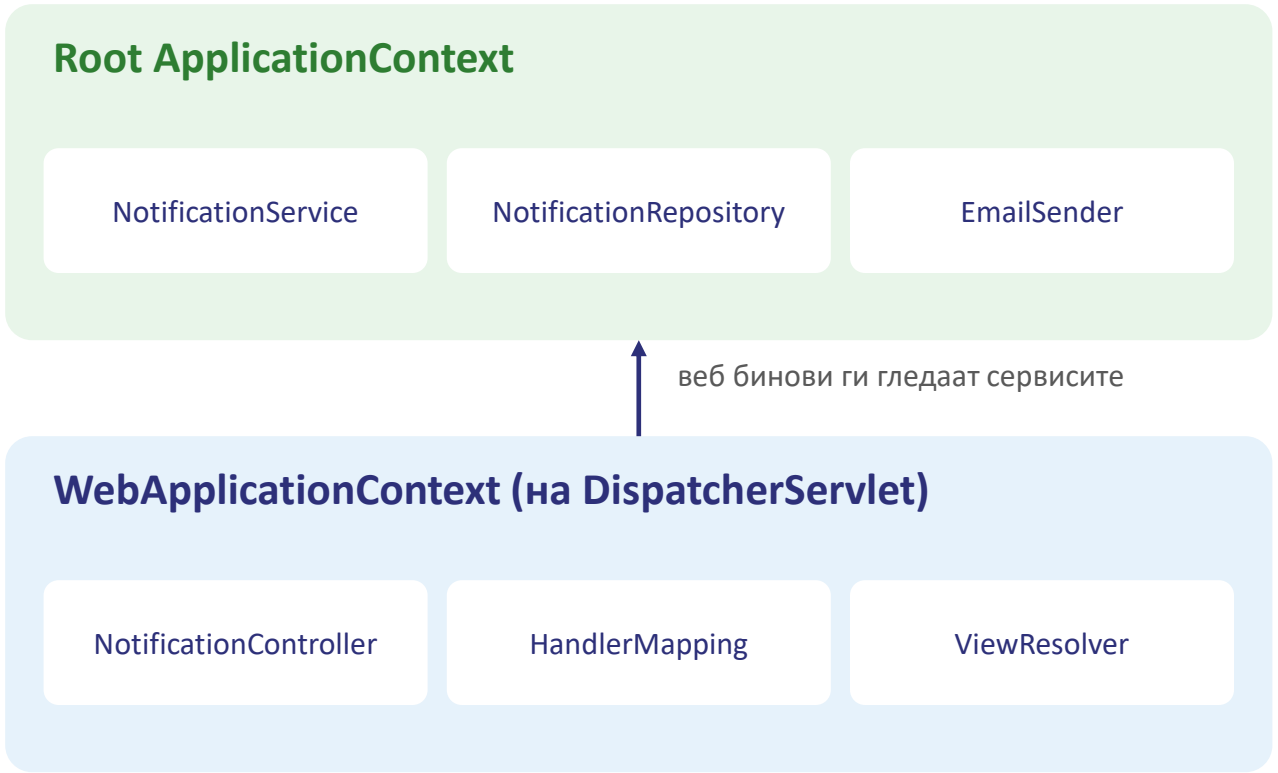
Ние го пишуваме само чекорот **4** (и шаблонот). Сè друго е Spring MVC.

Специјални бинови на DispatcherServlet

Бин	Прашање на кое одговара	Пример
`HandlerMapping`	Кој метод го обработува барањето?	`GET /welcome` → `showPage()`
`HandlerAdapter`	Како да го повикам методот?	ги разрешува аргументите
`ViewResolver`	Кој шаблон одговара на името?	`"index"` → `templates/index.html`
`HandlerExceptionResolver`	Што ако се фрли исклучок?	исклучок → статус 404 / страница
`LocaleResolver`	На кој јазик е клиентот?	`Accept-Language: mk`
`MultipartResolver`	Има ли прикачена датотека?	форма за upload

Не ги пишуваме ние — **Spring Boot ги регистрира автоматски**. Ние пишуваме контролери.

WebApplicationContext



- Проширување на **ApplicationContext** (IoC контејнер) за веб
- Има референца до **ServletContext**
- Го креира **DispatcherServlet**
- Повеќе DispatcherServlet-и (пр. `/web/*`, `/api/*`) → секој свој веб контекст, **заеднички** root
- Во Spring Boot: најчесто **еден** контекст

Прв контролер

WelcomeController.java

```java
@Controller
public class WelcomeController {

    @GetMapping("/welcome")
    public String showPage(Model model) {
        model.addAttribute("message",
                           "Здраво, ФИНКИ!");
        return "index";
    }
}
```

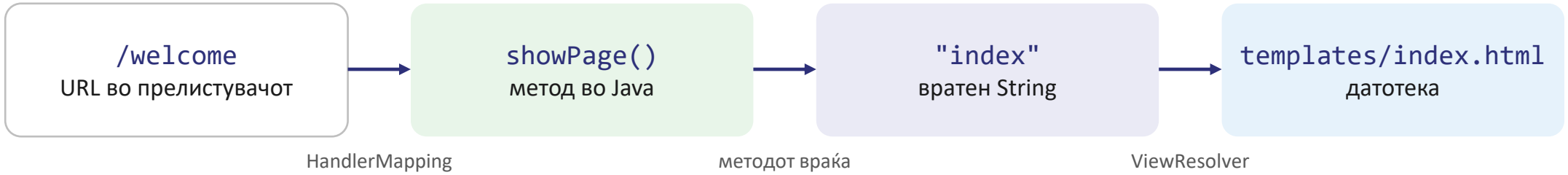
resources/templates/index.html

```html
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<body>
  <h1 th:text="${message}">текст</h1>
</body>
</html>
```

- `@Controller` — бин + веб контролер
- `@GetMapping("/welcome")` — URL што го обработува
- `Model` — Spring го креира и го предава
- `return "index"` — **логичко име** на преглед

`th:text="${message}"` → текстот се заменува со атрибутот **message** од моделот

Три имиња — три различни работи



Име	Каде е	Кој го користи	Мора да се совпаѓа со
`/welcome`	`@GetMapping("/welcome")`	HandlerMapping	URL-то во барањето
`showPage`	име на методот	**никој** — Spring не го гледа	— (слободно избираме)
`"index"`	`return "index";`	ViewResolver	`templates/index.html`
`"message"`	`model.addAttribute("message", …)`	Thymeleaf	`${message}` во шаблонот

URL, метод и шаблон **не мора** да имаат исто име — секое име го користи различна компонента.

GET /welcome: чекор по чекор

#	Кој	Што се случува
1	DispatcherServlet	го прима `GET /welcome`
2	HandlerMapping	`GET /welcome` → `WelcomeController.showPage(Model)`
3	HandlerAdapter	креира празен `Model`, го повикува `showPage(model)`
4	WelcomeController	`message = "Здраво, ФИНКИ!"`, враќа `"index"`
5	ViewResolver	`"index"` → `classpath:/templates/index.html`
6	Thymeleaf	`${message}` → текст → HTML во телото на одговорот

Прелистувачот добива:

```
<!DOCTYPE html>
<html>
<body>
  <h1>Здраво, ФИНКИ!</h1>
</body>
</html>
```

Нема `th:` атрибути — само **чист HTML**

Thymeleaf и ViewResolver



Што е Thymeleaf?

Систем за шаблони што работи на **серверот**

шаблон + модел → HTML

Шаблонот е валиден HTML; `th:` атрибутите ги обработува серверот

Прелистувачот добива **чист HTML**

Правило	Значење	Пример
`xmlns:th`	овозможува `th:` атрибути	`xmlns:th="http://www.thymeleaf.org"`
`${…}`	атрибут од моделот	`${message}, ${n.to}`
`th:text`	го заменува текстот	`<td th:text="${n.to}">…</td>`
`th:each`	повторува за секој елемент	`<tr th:each="n : ${notifications}">`
`th:if`	прикажи само ако е точно	`<p th:if="${info}" th:text="${info}">`
`@{…}`	гради линк (URL)	`@{/notifications/{id}(id=${n.id})}`

Листа на известувања како HTML

NotificationController

```
@GetMapping("/notifications")
public String list(Model model) {
    model.addAttribute("notifications",
                       service.findAll());
    return "notification-list";
}
```

localhost:8080/notifications

До	Порака
ana@finki.ukim.mk	Добредојде!
marko@finki.ukim.mk	Испит во петок

templates/notification-list.html

```
<table>
  <tr><th>До</th><th>Порака</th></tr>
  <tr th:each="n : ${notifications}">
    <td th:text="${n.to}">ana@...</td>
    <td th:text="${n.message}">...</td>
  </tr>
</table>
```

- `th:each` — повторува ред за секој елемент
- `${n.to}` — компонента на record-от
- Текстот „ana@...“ се гледа само ако ја отвориш датотеката без сервер

@Controller или @RestController?

HTML страница

```
@Controller
public class NotificationController {
  @GetMapping("/notifications")
  public String list(Model model) {
    model.addAttribute("notifications",
                       service.findAll());
    return "notification-list"; // преглед
  }
}
```

→ templates/notification-list.html → HTML

JSON податоци

```
@RestController
public class NotificationRestController {
  @GetMapping("/api/notifications")
  public List<Notification> list() {
    return service.findAll();  // податоци
  }
}
```

```
[{"id":1,"to":"ana@finki.ukim.mk",
  "message":"Добредојде!","channel":"email"}]
```

`@RestController` = `@Controller` + `@ResponseBody` → објектот се претвора во **JSON** (Jackson)

@RequestMapping: кога се повикува методот?

Својство	Пример	Се совпаѓа кога…
`path / value`	`@RequestMapping("/notifications")`	патеката е `/notifications`
`method`	`method = RequestMethod.POST`	HTTP методот е POST
`params`	`params = "channel=sms"`	има параметар `channel=sms`
`headers`	`headers = "X-Api-Version=2"`	заглавјето ја има таа вредност
`consumes`	`consumes = "application/json"`	`Content-Type` на барањето е JSON
`produces`	`produces = "application/json"`	клиентот прифаќа JSON (`Accept`)

Сите наведени услови мора да важат **истовремено**. Без `method` → важи за **сите** HTTP методи.

Мапирање на класа + на метод

```java
@Controller
@RequestMapping("/notifications")          // префикс за сите методи
public class NotificationController {

    @GetMapping                            // GET  /notifications
    public String list(Model model) { ... }

    @GetMapping("/{id}")                   // GET  /notifications/7
    public String details(@PathVariable Long id, Model model) { ... }

    @GetMapping("/new")                    // GET  /notifications/new
    public String form() { ... }

    @PostMapping                           // POST /notifications
    public String create(...) { ... }
}
```

Класа

Заеднички **префикс**

Метод

HTTP метод + **остаток** од патеката

Финална патека

класа + метод

`/notifications/new` или `/{id}`? Победува **поконкретната** патека → `form()`

Кратенки според HTTP методот

Анотација	Исто како	Во нашата апликација
`@GetMapping`	`@RequestMapping(method = GET)`	прикажи листа / едно известување
`@PostMapping`	`@RequestMapping(method = POST)`	креирај ново известување
`@PutMapping`	`@RequestMapping(method = PUT)`	замени цело известување
`@PatchMapping`	`@RequestMapping(method = PATCH)`	измени дел (пр. само каналот)
`@DeleteMapping`	`@RequestMapping(method = DELETE)`	избриши известување

`@RequestMapping` го користиме **на класата** (префикс) или кога методот прифаќа **повеќе** HTTP методи.

Чести грешки и што значат

Што гледаш	Најчеста причина
`404 Whitelabel Error Page`	нема мапирање за URL-то, или контролерот е надвор од пакетот на `@SpringBootApplication`
`405 Method Not Allowed`	URL-то постои, но за друг HTTP метод (GET наместо POST)
`400 Bad Request`	недостасува задолжителен параметар или погрешен тип (`abc` → `Long`)
`401 Unauthorized`	не си најавен — вклучен е Spring Security, а барањето нема најава
`403 Forbidden`	најавен си, но немаш право (улога)
`415 Unsupported Media Type`	`@RequestBody` без `Content-Type: application/json`
`500 TemplateInputException`	вратено име на преглед за кое нема шаблон во `templates/`
`500 кај метод што враќа објект`	`@Controller` наместо `@RestController` (нема `@ResponseBody`)

Предвиди: кој статус ќе се врати?

```
@Controller
@RequestMapping("/notifications")
class NotificationController {

  @GetMapping("/{id}")
  public String details(
      @PathVariable Long id, Model m) {...}

  @PostMapping
  public String create(
      @RequestParam String to,
      @RequestParam(required = false)
          String channel) {...}
}
```

1. GET /notifications/1
2. GET /notifications/-123
3. GET /notifications/abc
4. GET /notifications/5/details
5. GET /notifications
6. POST /notifications?to=ana@finki.ukim.mk
7. POST /notifications

Одговор: статуси

1	`GET /notifications/1`	200	id = 1 (OK — успешно)
2	`GET /notifications/-123`	200	-123 е валиден Long (OK — успешно)
3	`GET /notifications/abc`	400	„abc“ не е Long (Bad Request — неисправно барање)
4	`GET /notifications/5/details`	404	нема мапирање со два сегменти (Not Found — не е пронајдено)
5	`GET /notifications`	405	/notifications е само за POST (Method Not Allowed — методот не е дозволен)
6	`POST /notifications?to=ana@finki.ukim.mk`	200	channel е опционален → null (OK — успешно)
7	`POST /notifications`	400	недостасува задолжителен to (Bad Request — неисправно барање)

Резиме на Час 1

MVC за веб

Секоја акција = ново HTTP барање; контролерот избира модел и преглед

Слоеви

Веб → Сервис → Податочен пристап; моделот без зависности

DispatcherServlet

Еден сервлет за сите барања; делегира до методите

Специјални бинови

HandlerMapping, HandlerAdapter, ViewResolver, ExceptionResolver

@Controller

String = име на преглед → Thymeleaf → HTML

@RestController

Објект → JSON; @RequestMapping / @GetMapping за мапирање

Податоци од барањето и одговори

Аргументи, повратни вредности, грешки и REST

1 — 2

Час 1 **Час 2**

Универзитет „Св. Кирил и Методиј“ во Скопје
ФАКУЛТЕТ ЗА ИНФОРМАТИЧКИ НАУКИ
И КОМПЈУТЕРСКО ИНЖЕНЕРСТВО

Цели

Потсетник

- DispatcherServlet ги прима сите барања
- HandlerMapping: барање → метод
- `@Controller` → преглед (HTML)
- `@RestController` → JSON
- `@RequestMapping`, `@GetMapping`, `@PostMapping`

Денес ќе научиме

1. Делови од барањето → **аргументи** на методот
2. `@PathVariable`, `@RequestParam`, `@RequestHeader`
3. Форми, пренасочување и **сесија**
4. JSON и `@RequestBody`, конверзија на типови
5. Повратни вредности, `ResponseEntity`, грешки
6. **REST** API за известувањата

Од каде доаѓаат аргументите?

```
POST /api/notifications/7/resend?channel=sms HTTP/1.1
Host: localhost:8080
Accept-Language: mk
Cookie: theme=dark
Content-Type: application/json

{"to": "ana@finki.ukim.mk", "message": "Потсетник"}
```

Дел од барањето	Во примерот	Аргумент на методот
Патека	`/7/`	`@PathVariable Long id`
Query параметар	`?channel=sms`	`@RequestParam String channel`
Заглавје	`Accept-Language: mk`	`@RequestHeader("Accept-Language") String lang`
Колаче	`theme=dark`	`@CookieValue String theme`
Тело (JSON)	`{"to": ..., "message": ...}`	`@RequestBody NotificationRequest req`

Шаблони за патеки (PathPattern)

Шаблон	Се совпаѓа	Не се совпаѓа
`/notifications/?`	`/notifications/1`	`/notifications/12`
`/files/*.txt`	`/files/a.txt`	`/files/a/b.txt`
`/files/**`	`/files`, `/files/a/b/c`	`/docs/a`
`/notifications/{id}`	`/notifications/7` (id = "7")	`/notifications/7/edit`
`/notifications/{id:\d+}`	`/notifications/7`	`/notifications/abc` → 404
`/channels/{ch:email|sms}`	`/channels/sms` (ch = "sms")	`/channels/fax` → 404
`/users/{name:[a-z]+}`	`/users/ana` (name = "ana")	`/users/Ana1` → 404
`/files/{*path}`	`/files/a/b` (path = "/a/b")	`/docs/a`

`**` и `{*path}` се дозволени **само на крајот** на патеката. Најчесто: `{id}`.

@PathVariable: вредност од патеката

```java
@GetMapping("/notifications/{id}")
public String details(@PathVariable Long id, Model model) {
    model.addAttribute("notification", service.findById(id)
            .orElseThrow(() -> new NotificationNotFoundException(id)));
    return "notification-details";
}

@GetMapping("/users/{userEmail}/notifications/{nid}")
public String forUser(@PathVariable("userEmail") String email,
                      @PathVariable Long nid, Model model) {
    ...
}
```

- Име на аргументот = име во `{ }`
- Различно име → `@PathVariable("userEmail")`
- `String` → `Long` автоматски
- `/notifications/abc` → **400**
- Нема `{id}` во патеката → **500** (наша грешка)

`GET /users/ana@finki.ukim.mk/notifications/3` → `email = "ana@finki.ukim.mk", nid = 3`

@RequestParam: параметри од барањето

```java
@GetMapping("/notifications/search")
public String search(
    @RequestParam String to,
    @RequestParam(defaultValue = "10") int limit,
    @RequestParam(required = false) String channel,
    Model model) {
  model.addAttribute("notifications",
      service.findByTo(to, channel, limit));
  return "notification-list";
}
```

- Стандардно **задолжителен**
- `required = false` или `Optional<String>` → незадолжителен
- `defaultValue` → незадолжителен, со предефинирана вредност
- Работи за **query string** и за полиња од **форма** (POST)

Барање	Резултат
`/notifications/search?to=ana`	`to="ana"`, `limit=10`, `channel=null`
`/notifications/search?to=ana&limit=3&channel=sms`	`limit=3`, `channel="sms"`
`/notifications/search`	**400** — нема `to`
`/notifications/search?to=ana&limit=пет`	**400** — „пет“ не е `int`

HTML форма → @RequestParam

```
<form action="/notifications/search" method="get">
  <input name="to" type="email">
  <select name="channel">
    <option value="email">Е-пошта</option>
    <option value="sms">SMS</option>
  </select>
  <input name="limit" type="number" value="10">
  <button type="submit">Барај</button>
</form>
```

Прелистувачот праќа:

```
GET /notifications/search
    ?to=ana%40finki.ukim.mk
    &channel=sms&limit=10
```

- `name="to"` → `@RequestParam String to`
- `name="limit"` → `@RequestParam int limit`
- `method="post"` → параметрите се во **телото**, но `@RequestParam` работи исто

Врската е **името**: `name` во формата = име на параметарот во методот.

@RequestHeader и @CookieValue

```java
@GetMapping("/welcome")
public String welcome(
        @RequestHeader("Accept-Language") String language,
        @RequestHeader(value = "User-Agent", required = false) String agent,
        @CookieValue(value = "theme", defaultValue = "light") String theme,
        Model model) {
    model.addAttribute("language", language);   // "mk"
    model.addAttribute("theme", theme);         // "dark" или "light"
    return "welcome";
}
```

Правила

Исти правила како `@RequestParam`: задолжителни, `required`, `defaultValue`, конверзија

Корисно

```java
@RequestHeader Map<String, String> all
@CookieValue("JSESSIONID") String id
```

Аргументи според типот

Тип на аргумент	Што вметнува Spring
`Model, ModelMap, Map`	моделот за прегледот (празен на почеток)
`HttpServletRequest, HttpServletResponse`	барањето и одговорот од Servlet API
`HttpSession`	сесијата — се креира ако не постои, **никогаш null**
`HttpMethod`	HTTP методот на барањето (GET, POST…)
`Locale, TimeZone, ZoneId`	јазикот и временската зона на клиентот
`InputStream / Reader, OutputStream / Writer`	суров влез / излез

```
@GetMapping("/debug")
public String debug(
    HttpMethod method,
    Locale locale,
    HttpSession session,
    Model model) {
  ...
}
```

Името на аргументот не е важно — важен е **типот**

Форма → објект: @ModelAttribute

```
<form action="/notifications" method="post">
  <input name="to">
  <textarea name="message"></textarea>
  <select name="channel">...</select>
  <button>Испрати</button>
</form>
```

- Spring го **креира** објектот
- Полињата се поврзуваат **по име**
- Објектот е и **атрибут во моделот**

```
public record NotificationRequest(String to, String message, String channel) { }

@PostMapping("/notifications")
public String create(@ModelAttribute NotificationRequest request) {
    service.notifyUser(request.to(), request.message(), request.channel());
    return "redirect:/notifications";
}
```

Ист резултат како `@RequestParam String to, @RequestParam String message` — но за **многу полиња**.

Прикачување датотека: MultipartFile

1. Форма — задолжително `enctype="multipart/form-data"`

```
<form th:action="@{/notifications/attachment}" method="post"
      enctype="multipart/form-data">
  <input type="file" name="file"> <button>Прикачи</button>
</form>
```

2. Контролер — името `file` = `name="file"` во формата

```
@PostMapping("/notifications/attachment")
public String upload(@RequestParam MultipartFile file) throws IOException {
    if (file.isEmpty()) return "redirect:/notifications";   // ништо не е избрано
    Path dir = Files.createDirectories(Path.of("uploads"));
    Files.copy(file.getInputStream(), dir.resolve(file.getOriginalFilename()),
               StandardCopyOption.REPLACE_EXISTING);
    return "redirect:/notifications";
}
```

MultipartFile	Пример
`getOriginalFilename()`	`"izvestaj.pdf"`
`getSize()`	`52340` бајти
`getContentType()`	`"application/pdf"`
`isEmpty()`	нема датотека
`getInputStream()`	содржината

Повеќе датотеки: `<input type="file" name="files" multiple>` → `@RequestParam List<MultipartFile> files`
Големина: стандардно до 1 MB по датотека → `spring.servlet.multipart.max-file-size=10MB`

Без `enctype="multipart/form-data"` прелистувачот праќа само **името** на датотеката, не и содржината

По POST: пренасочи (redirect)

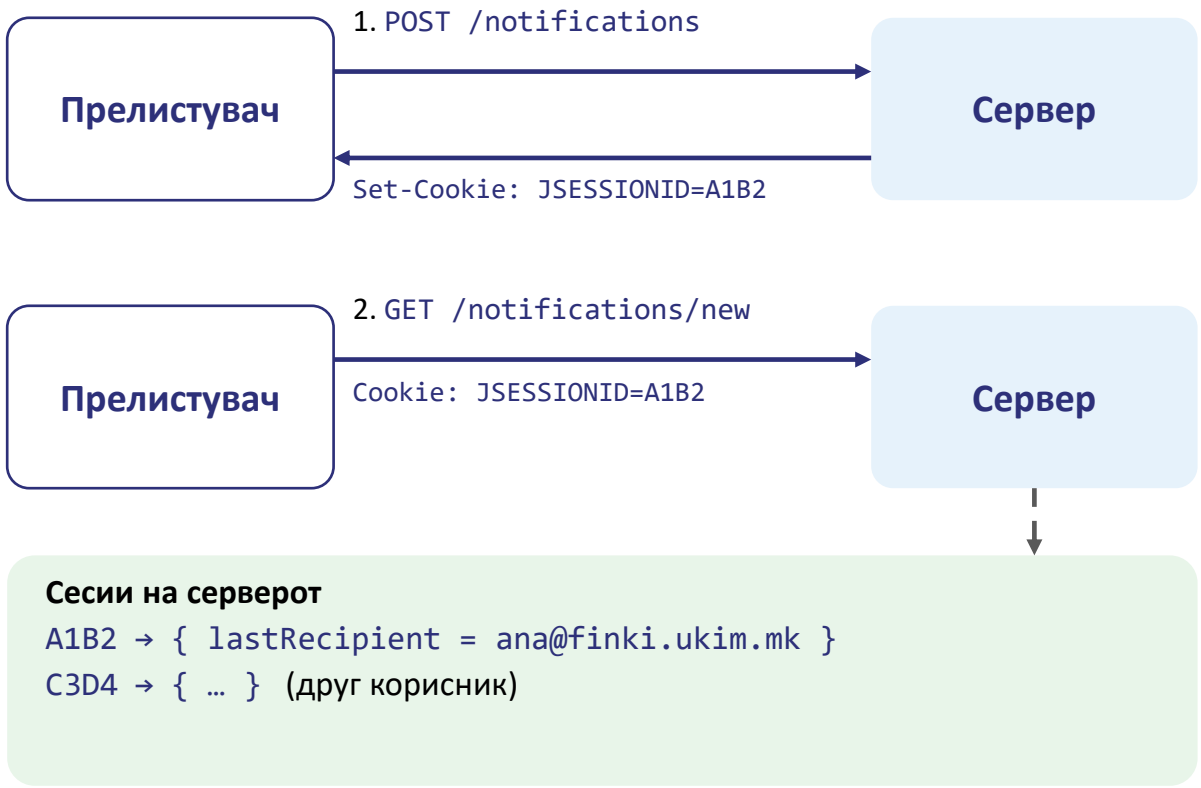
1. `POST /notifications`
   формата се испраќа
2. `302 Found`
   `Location: /notifications`
3. `GET /notifications`
   прелистувачот сам го праќа
4. `200 OK`
   HTML со листата

```
@PostMapping
public String create(@ModelAttribute NotificationRequest request,
                     RedirectAttributes redirect) {
    service.notifyUser(request.to(), request.message(), request.channel());
    return "redirect:/notifications";
}
```

- `redirect:` → **302** + `Location`
- F5 не го повторува POST-от

Без redirect: **F5** = повторно POST = **дупликат известување**

Сесија: податоци меѓу барањата



- HTTP е **без состојба** — секое барање е ново
- Сесијата е мапа на **серверот**, посебна за **секој корисник**
- Се препознава преку колачето **JSESSIONID**

Во Spring MVC

- `HttpSession` како аргумент
- `@SessionAttribute` — читање

Истата `HttpSession` од сервлетите — Spring само дава **поудобен пристап** до неа.

HttpSession и @SessionAttribute

```java
@PostMapping
public String create(@ModelAttribute NotificationRequest req,
                     HttpSession session) {
    service.notifyUser(req.to(), req.message(), req.channel());
    session.setAttribute("lastRecipient", req.to());    // запиши
    return "redirect:/notifications";
}

@GetMapping("/new")
public String form(
        @SessionAttribute(required = false) String lastRecipient,
        Model model) {
    model.addAttribute("to", lastRecipient);            // прочитај
    return "notification-form";
}
```

```html
<input name="to" th:value="${to}">   <!-- веќе пополнето -->
```

HttpSession

Се креира ако не постои — **никогаш null**

`setAttribute`, `getAttribute`, `invalidate`

@SessionAttribute

Само **чита** постоечки атрибут

Име на аргументот = име на атрибутот

Го нема → **400**, освен `required = false`

Колку долго живеат податоците?

Каде	Колку трае	Пример
`Model атрибут`	**едно** барање	`model.addAttribute(…)`
`Flash атрибут`	до **следното** барање (по redirect)	`redirect.addFlashAttribute("info", …)`
`HttpSession`	додека трае сесијата (истек, одјава)	`lastRecipient`
`Singleton бин`	додека работи апликацијата — **за сите корисници**	`NotificationRepository`

Податоци на **еден корисник** → сесија. **Никогаш** во singleton бин — тој е заеднички за сите.

JSON во телото: @RequestBody

```
POST /api/notifications HTTP/1.1
Content-Type: application/json

{"to": "ana@finki.ukim.mk", "message": "Добредојде!"}
```

```java
@RestController
@RequestMapping("/api/notifications")
public class NotificationRestController {

    @PostMapping
    public Notification create(@RequestBody NotificationRequest request) {
        return service.notifyUser(request.to(), request.message(),
                                  request.channel());   // null → "email"
    }
}
```

- JSON → Java преку **HttpMessageConverter** (Jackson)
- Без JSON `Content-Type` → **415**
- Невалиден JSON → **400**

`@ModelAttribute` = полиња од **форма** · `@RequestBody` = **JSON** во телото

Конверзија на типови

Во барањето (String)	Аргумент	Резултат
`"7"`	`Long id`	`7`
`"true"`	`boolean urgent`	`true`
`"SMS"`	`Channel channel  (enum)`	`Channel.SMS`
`"2026-11-03"`	`@DateTimeFormat(iso = DATE)` `LocalDate day`	`2026-11-03`
`"abc"`	`Long id`	`400 Bad Request`

Од HTTP барањето секогаш добиваме **текст** — Spring го претвора во типот на аргументот.

Сопствени типови

Converter

еден тип → друг тип

Formatter

`String` ↔ тип (двонасочно, со `Locale`)

Property editors

стар начин — не се препорачува

@DateTimeFormat: текст → датум

```
@GetMapping("/notifications/search")       // ?from=2026-11-03
public String search(
        @RequestParam @DateTimeFormat(iso = ISO.DATE) LocalDate from,
        Model model) {
    model.addAttribute("notifications", service.findSince(from));
    return "notification-list";
}
```

- Датумот нема еден формат: `2026-11-03`, `03.11.2026`, `11/3/26`
- Анотацијата му кажува на Spring **кој формат** да го чита
- Без неа или со погрешен формат → **400**

Во барањето	Анотација	Тип	Типичен извор
`"2026-11-03"`	`@DateTimeFormat(iso = ISO.DATE)`	`LocalDate`	`<input type="date">`
`"03.11.2026"`	`@DateTimeFormat(pattern = "dd.MM.yyyy")`	`LocalDate`	рачно внесен текст
`"2026-11-03T14:30"`	`@DateTimeFormat(iso = ISO.DATE_TIME)`	`LocalDateTime`	`<input type="datetime-local">`

Глобално, за сите параметри: `spring.mvc.format.date=dd.MM.yyyy` во `application.properties`

Внимание: за JSON (`@RequestBody`) не важи — таму Jackson: `@JsonFormat(pattern = "dd.MM.yyyy")`

Други корисни конверзии

Во барањето	Аргумент	Резултат
`?channels=email,sms`	`@RequestParam List<String> channels`	`[email, sms]`
`?channel=email&channel=sms`	`@RequestParam List<String> channel`	`[email, sms]`
`?channel=sms`	`@RequestParam Channel channel` (enum)	`400` — мора точно `"SMS"`
параметарот го нема	`@RequestParam Optional<Integer> limit`	`Optional.empty()`
`?rate=15%`	`@NumberFormat(style = Style.PERCENT) double rate`	`0.15`

Листа: со запирка **или** со повторен параметар · `enum`: разликува големи и мали букви · `@NumberFormat` е „братот“ на `@DateTimeFormat` за броеви

Сите одат преку истиот механизам (`ConversionService`) — за сопствени типови: **`Converter`** / **`Formatter`**

Што може да врати методот?

Повратна вредност	Што прави Spring	Пример
`String`	име на преглед → ViewResolver	`return "notification-list";`
`"redirect:…"`	302 пренасочување	`return "redirect:/notifications";`
`ModelAndView`	преглед + модел во еден објект	`new ModelAndView("notification-list")`
`објект (@ResponseBody)`	објект → JSON во телото	`return service.findAll();`
`ResponseEntity<T>`	тело + статус + заглавја	`ResponseEntity.created(uri).body(n)`
`void`	нема тело (пр. со `@ResponseStatus`)	`void delete(@PathVariable Long id)`

`@Controller` → најчесто **String** · `@RestController` → **објект** или **ResponseEntity**

ResponseEntity: статус, заглавја, тело

```java
@GetMapping("/{id}")
public ResponseEntity<Notification> findById(
        @PathVariable Long id) {
    return service.findById(id)
            .map(ResponseEntity::ok)                          // 200
            .orElseGet(() -> ResponseEntity.notFound().build()); // 404
}

@PostMapping
public ResponseEntity<Notification> create(
        @RequestBody NotificationRequest req) {
    Notification n = service.notifyUser(
            req.to(), req.message(), req.channel());
    URI uri = URI.create("/api/notifications/" + n.id());
    return ResponseEntity.created(uri).body(n);              // 201
}
```

Одговор на POST:

```
HTTP/1.1 201 Created
Location: /api/notifications/8
Content-Type: application/json

{"id":8,"to":"ana@finki.ukim.mk",
 "message":"Добредојде!",
 "channel":"email"}
```

`created(uri)` прави две работи:

- статус **201 Created** = креиран е **нов ресурс**
- заглавје `Location` = адресата на новиот ресурс

`.body(n)` — го додава телото:

- известувањето како JSON (горе)

`ok()` → 200 · `created(uri)` → 201 + `Location` · `notFound()` → 404 · `badRequest()` → 400

@ResponseBody или ResponseEntity?

Во **@Controller** — со и без **@ResponseBody**

```
@Controller
public class NotificationController {
    @GetMapping("/notifications")          // без @ResponseBody
    public String list(Model model) {
        model.addAttribute("notifications", service.findAll());
        return "notification-list";        // → име на преглед → HTML
    }

    @GetMapping("/notifications/count")
    @ResponseBody                          // со @ResponseBody
    public int count() {
        return service.findAll().size();   // → тело: 3
    }

    @GetMapping("/notifications/data")
    @ResponseBody
    public List<Notification> data() {
        return service.findAll();          // → тело: JSON листа
    }
}
```

	@ResponseBody	ResponseEntity
Тело	вратениот објект	`.body(n)`
Статус	секогаш **200**	ние избираме: `201`, `404`
Заглавја	не	да (`Location`)
Кога?	одговорот е секогаш ист вид	статусот зависи од ситуацијата

`@RestController` = `@Controller` + `@ResponseBody` на секој метод

`String` без `@ResponseBody` = **име на преглед** · со `@ResponseBody` = **самиот текст** во одговорот

Грешки: два начина од исклучок до статус

1. @ResponseStatus на исклучокот

`findById` само фрла — статусот е запишан на класата

```
@ResponseStatus(HttpStatus.NOT_FOUND)          // → 404
public class NotificationNotFoundException
        extends RuntimeException {
    public NotificationNotFoundException(Long id) {
        super("Известување " + id + " не постои");
    }
}

// контролер: само фрла
@GetMapping("/{id}")
public Notification findById(@PathVariable Long id) {
    return service.findById(id).orElseThrow(
            () -> new NotificationNotFoundException(id));
}
```

`GET /api/notifications/99` → `findById` фрла → Spring ја чита `@ResponseStatus` → **404**

2. @ExceptionHandler метод

`badInput` ја фаќа грешката и го гради одговорот

```
// сервис: фрла
if (message.length() > 160)
    throw new IllegalArgumentException("Предолга порака");

// контролер: create() не фаќа ништо...
@PostMapping
public ResponseEntity<Notification> create(...) {
    ... service.notifyUser(...) ...     // ← тука се фрла
}
// ...овој метод го фаќа
@ExceptionHandler(IllegalArgumentException.class)   // → 400
public ResponseEntity<String> badInput(
        IllegalArgumentException e) {
    return ResponseEntity.badRequest().body(e.getMessage());
}
```

`POST` со порака од 200 знаци → сервисот фрла → Spring го повикува `badInput(e)` → **400** + „Предолга порака“

`@ResponseStatus`: исклучокот го носи статусот · **`@ExceptionHandler`**: ние го градиме одговорот

@ExceptionHandler: чекор по чекор

#	Што се случува
1	клиентот праќа `POST /api/notifications` со порака од 200 знаци
2	`create()` го повикува `service.notifyUser(...)`
3	сервисот фрла `IllegalArgumentException("Предолга порака")`
4	`create()` не ја фаќа → исклучокот излегува од методот
5	Spring бара `@ExceptionHandler` за тој тип **во истиот контролер** → `badInput(e)`
6	клиентот добива **400 Bad Request**, тело: `Предолга порака`

Ако...	Статус
ништо не правиме	500
`@ResponseStatus` на исклучокот	тој статус
`@ExceptionHandler` метод	ние одлучуваме
`@ControllerAdvice` класа	за сите контролери

За сите контролери одеднаш:

```
@ControllerAdvice
public class GlobalErrors {
    @ExceptionHandler(...)
    ... badInput(...) { ... }
}
```

Сите три ги извршува `HandlerExceptionResolver` — ако нема ниту едно, клиентот добива **500**

REST: ресурси наместо страници

✓ Добро	✗ Лошо	Правило
`/api/notifications`	`/api/getNotifications`	именка — акцијата е HTTP методот
`/api/notifications/7`	`/api/notification?id=7`	множина + хиерархија
`/api/shopping-carts`	`/api/Shopping_Carts`	мали букви, цртичка
`/api/notifications`	`/api/notifications/`	без коса црта на крајот
`/api/notifications/7  + Accept`	`/api/notifications/7.json`	без екстензии

Ресурсот е **именка во множина**; што правиме со него кажува **HTTP методот**.

REST контролер за известувања

```
@RestController
@RequestMapping("/api/notifications")
public class NotificationRestController {
    private final NotificationService service;

    public NotificationRestController(
            NotificationService service) {
        this.service = service;
    }

    @GetMapping
    public List<Notification> findAll() {
        return service.findAll();
    }

    @GetMapping("/{id}")
    public Notification findById(@PathVariable Long id) {
        return service.findById(id).orElseThrow(
            () -> new NotificationNotFoundException(id));
    }

    @PostMapping
    public ResponseEntity<Notification> create(
            @RequestBody NotificationRequest req) {
        Notification n = service.notifyUser(
                req.to(), req.message(), req.channel());
        URI uri = URI.create("/api/notifications/" + n.id());
        return ResponseEntity.created(uri).body(n);
    }

    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void delete(@PathVariable Long id) {
        service.deleteById(id);
    }
}
```

Пробај го API-то

```
$ curl http://localhost:8080/api/notifications/1
{"id":1,"to":"ana@finki.ukim.mk","message":"Добредојде!","channel":"email"}

$ curl -i -X POST http://localhost:8080/api/notifications \
    -H "Content-Type: application/json" \
    -d '{"to":"marko@finki.ukim.mk","message":"Испит во петок"}'
HTTP/1.1 201
Location: /api/notifications/2

$ curl -i -X DELETE http://localhost:8080/api/notifications/2
HTTP/1.1 204

$ curl -i http://localhost:8080/api/notifications/99
HTTP/1.1 404
```

`-i`
include: прикажи ги и статусот и заглавјата

`-X`
HTTP метод (без него: GET)

`-H`
додај заглавје (header)

`-d`
тело (data) само по себе → POST

Алатки

- **curl** (терминал)
- IntelliJ **HTTP Client** (`.http`)
- **Postman**

Прелистувач: само **GET**

`\` на крајот од редот: командата продолжува во следниот ред

Користење на туѓ REST API: RestTemplate

```java
@Configuration
public class RestConfig {
    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }
}

@Service
public class NotificationClient {
    private static final String API =
            "http://localhost:8080/api/notifications";
    private final RestTemplate rest;
    public NotificationClient(RestTemplate rest) { this.rest = rest; }

    public Notification get(Long id) {
        return rest.getForObject(API + "/{id}",
                                 Notification.class, id);
    }
    public URI send(NotificationRequest req) {
        return rest.postForLocation(API, req);     // Location од 201
    }
}
```

Метод	HTTP
`getForObject`	GET → објект
`getForEntity`	GET → ResponseEntity
`postForObject`	POST → објект
`postForLocation`	POST → URI
`put / delete`	PUT / DELETE
`exchange`	било кој метод

Поново: **RestClient**, **WebClient** (додатен материјал)

RestTemplate: што се случува при повик?

`rest.getForObject(API + "/{id}", Notification.class, 7)`

#	Што се случува
1	`{id}` се заменува со `7` → `GET …/api/notifications/7`
2	барањето се праќа и методот **чека** одговор (синхроно)
3	серверот (нашиот REST контролер) враќа `200` + JSON
4	Jackson: JSON → `Notification` — како `@RequestBody`, само обратно
5	ако серверот врати `404` → исклучок `HttpClientErrorException.NotFound`

Треба и статусот? getForEntity

```
ResponseEntity<Notification> r =
    rest.getForEntity(API + "/{id}",
                      Notification.class, 7);
r.getStatusCode();      // 200 OK
r.getBody();            // Notification
```

Грешка од серверот

```
try {
    return rest.getForObject(API + "/{id}",
                             Notification.class, id);
} catch (HttpClientErrorException.NotFound e) {
    return null;                    // го нема
}
```

Тука нашата апликација е **клиент** — ги прима статусите и заглавјата што ги праќа серверот

Резиме на темата

Барање → метод

DispatcherServlet, HandlerMapping, `@GetMapping`, `@PostMapping`…

Барање → аргументи

`@PathVariable`, `@RequestParam`, `@RequestHeader`, `@CookieValue`

Форми, сесија, JSON

`@ModelAttribute`, `HttpSession`, `addFlashAttribute`, `@RequestBody`

Метод → одговор

String (преглед), `redirect:`, објект → JSON, `ResponseEntity`

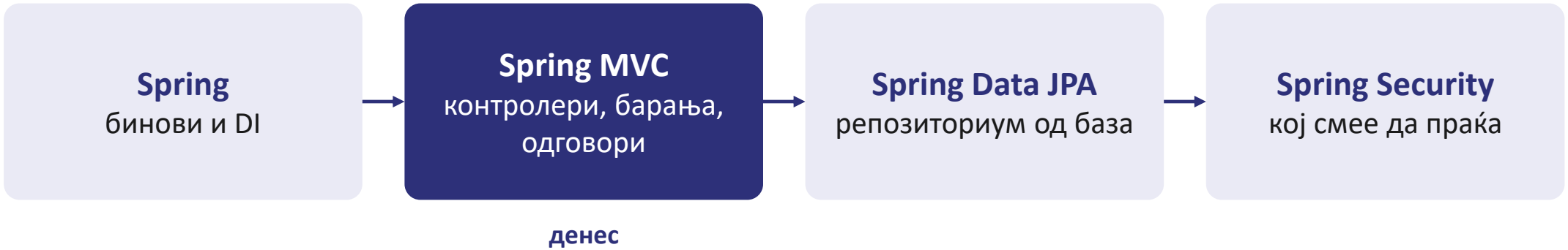
Грешки

`@ResponseStatus`, `@ExceptionHandler`, `@ControllerAdvice`

REST

ресурси во множина, HTTP методи, статус кодови 200/201/204/4xx

Што следува?



Слоевите **Controller → Service → Repository** остануваат — следно се менува само репозиториумот.

Додатен материјал

За самостојно читање

Пресретнувачи (HandlerInterceptor)

```
public class TimingInterceptor implements HandlerInterceptor {
    @Override
    public boolean preHandle(HttpServletRequest req, HttpServletResponse resp,
                             Object handler) {
        req.setAttribute("start", System.currentTimeMillis());
        return true;                                    // false → прекини
    }

    @Override
    public void afterCompletion(HttpServletRequest req, HttpServletResponse resp,
                                Object handler, Exception ex) {
        long start = (long) req.getAttribute("start");
        System.out.println(req.getRequestURI() + ": "
                + (System.currentTimeMillis() - start) + " ms");
    }
}

@Configuration
public class WebConfig implements WebMvcConfigurer {
    @Override
    public void addInterceptors(InterceptorRegistry registry) {
        registry.addInterceptor(new TimingInterceptor())
                .addPathPatterns("/notifications/**");
    }
}
```

preHandle

пред методот; `false` го прекинува барањето

postHandle

по методот, пред прегледот

afterCompletion

секогаш на крај, обратен редослед

Целосна обработка на барање

1. LocaleResolver: јазик на барањето
2. RequestContextHolder: тековно барање
3. MultipartResolver: прикачени датотеки?
4. HandlerMapping → HandlerExecutionChain (нема → 404)
5. Пресретнувачи: preHandle
6. HandlerAdapter → метод на контролерот
7. Пресретнувачи: postHandle
8. Исклучок? → HandlerExceptionResolver (нерешен → 500)
9. ViewResolver → View → исцртување
10. afterCompletion, RequestHandledEvent

Повеќе јазици: LocaleResolver

1. Текстови по јазик во `src/main/resources/`

```
# messages.properties       (стандардна, мора да постои)
welcome=Добредојде
# messages_en.properties
welcome=Welcome
```

2. Во шаблонот: `#{...}` (не ${...})

```
<h1 th:text="#{welcome}">Добредојде</h1>
<a href="?lang=mk">MK</a> | <a href="?lang=en">EN</a>
```

LocaleResolver	Јазикот го зема од
`AcceptHeaderLocaleResolver`	**Accept-Language** (стандарден)
`SessionLocaleResolver`	сесијата — избор на корисникот
`CookieLocaleResolver`	колаче — се памети подолго

3. Избор на јазик со `?lang=en`

```
@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Bean                    // името мора да е localeResolver
    public LocaleResolver localeResolver() {
        SessionLocaleResolver r = new SessionLocaleResolver();
        r.setDefaultLocale(Locale.forLanguageTag("mk"));
        return r;
    }

    @Override
    public void addInterceptors(InterceptorRegistry registry) {
        LocaleChangeInterceptor i = new LocaleChangeInterceptor();
        i.setParamName("lang");                    // ?lang=en
        registry.addInterceptor(i);
    }
}
```

`?lang=en` → `LocaleChangeInterceptor` го чита → `SessionLocaleResolver` го памети во сесијата → `#{welcome}` = „Welcome“

`Locale` = јазик (+ земја): `mk`, `en_US` · Без конфигурација јазикот го одредува прелистувачот

Останати аргументи на методите

Аргумент	Намена	Пример
`@RequestAttribute`	атрибут поставен од филтер/пресретнувач	`@RequestAttribute Long start`
`HttpEntity<B>`	тело + заглавја на барањето	`HttpEntity<NotificationRequest> e`
`MultipartFile`	прикачена датотека	`@RequestParam MultipartFile file`
`@RequestPart`	дел од multipart/form-data	`@RequestPart("meta") Meta meta`
`Errors / BindingResult`	грешки при поврзување/валидација	веднаш по `@ModelAttribute` аргументот

Порака по redirect: Model се губи

✗ POST со Model — пораката се губи

```
@PostMapping("/notifications")
public String create(..., Model model) {
    service.notifyUser(...);
    model.addAttribute("info",
            "Известувањето е испратено");
    return "redirect:/notifications";
}
```

redirect = **ново** GET барање → `info` од POST-от се губи

✓ POST со Flash — пораката стигнува

```
@PostMapping("/notifications")
public String create(...,
                RedirectAttributes redirect) {
    service.notifyUser(...);
    redirect.addFlashAttribute("info",
            "Известувањето е испратено");
    return "redirect:/notifications";
}
```

Spring ја чува во **сесијата** и ја става во `Model` на GET-от

↓ потоа: `GET /notifications` — без промени

```
@GetMapping("/notifications")
public String list(Model model) {
    // со Flash: Spring веќе ја ставил info во model
    model.addAttribute("notifications", service.findAll());
    return "notification-list";
}
```

↓ шаблон: `notification-list.html`

```
<!-- се прикажува само ако info постои -->
<p th:if="${info}"
   th:text="${info}">порака</p>
```

`Model` живее едно барање · **Flash** го преживува redirect-от и се прикажува **еднаш**

Flash атрибут: чекор по чекор

#	Барање	Сесија	Model	Корисникот гледа
1	`POST /notifications`	`info` се зачувува	—	—
2	`302` → redirect	`info` чека	—	—
3	`GET /notifications`	`info` се **вади и брише**	`info` е тука	✓ **Известувањето е испратено**
4	F5 → `GET`	празно	нема `info`	листа без порака

Зошто не рачно во `HttpSession`? → мора сами да ја избришеме, инаку пораката останува засекогаш
Зошто не во URL-то (`?info=…`)? → останува при F5 и секој може да го измисли

Flash = порака што го преживува redirect-от, се прикажува **еднаш**, па исчезнува

Сопствен конвертор (пример од книгата)

```
public record Price(Double amount, String currency) { }

public class StringToPriceConverter implements Converter<String, Price> {
    @Override
    public Price convert(String from) {            // "100 MKD"
        String[] data = from.split(" ");
        return new Price(Double.parseDouble(data[0]), data[1]);
    }
}

@Configuration
public class WebConfig implements WebMvcConfigurer {
    @Override
    public void addFormatters(FormatterRegistry registry) {
        registry.addConverter(new StringToPriceConverter());
    }
}
```

Употреба

```
@GetMapping("/products")
public String byPrice(
    @RequestParam Price price) {
  ...
}
```

`?price=100 MKD` →

`Price(100.0, "MKD")`

Model + String или ModelAndView?

Model + String

```
@GetMapping
public String list(Model model) {
    model.addAttribute("notifications",
                       service.findAll());
    return "notification-list";
}
```

ModelAndView

```
@GetMapping
public ModelAndView list() {
    var mav = new ModelAndView("notification-list");
    mav.addObject("notifications",
                  service.findAll());
    return mav;
}
```

Ист резултат: модел со `notifications` + преглед `notification-list.html`

Без Spring Boot: рачна конфигурација

ViewResolver за JSP

```
@Bean
public ViewResolver internalResourceViewResolver() {
    InternalResourceViewResolver bean =
            new InternalResourceViewResolver();
    bean.setViewClass(JstlView.class);
    bean.setPrefix("/WEB-INF/view/");
    bean.setSuffix(".jsp");
    return bean;
}
```

Контролер преку интерфејс

```
public class WelcomeController
        implements org.springframework.web.servlet.mvc.Controller {

    @Override
    public ModelAndView handleRequest(HttpServletRequest req,
                                      HttpServletResponse resp) {
        return new ModelAndView("index", "message", "Здраво!");
    }
}
// + SimpleUrlHandlerMapping: "/welcome" → welcomeController
```

Spring Boot ова го прави **автоматски**. Анотираните контролери: флексибилни методи, мапирањето е во класата.

RestClient: замена за RestTemplate

Истиот NotificationClient со `RestClient` (Spring 6.1+)

```
@Bean            // Spring Boot го дава RestClient.Builder
public RestClient restClient(RestClient.Builder builder) {
    return builder.baseUrl("http://localhost:8080/api/notifications")
                  .build();
}

public Notification get(Long id) {
    return rest.get()                          // GET
            .uri("/{id}", id)                  // …/notifications/7
            .retrieve()                        // испрати
            .onStatus(HttpStatusCode::is4xxClientError,
                (req, res) -> { throw new NotificationNotFoundException(id); })
            .body(Notification.class);         // JSON → објект
}

public URI send(NotificationRequest req) {
    return rest.post().body(req)               // објект → JSON
            .retrieve().toBodilessEntity()
            .getHeaders().getLocation();       // Location од 201
}
```

Споредба на истиот повик

```
// RestTemplate
rest.getForObject(API + "/{id}",
                  Notification.class, 7);
```

```
// RestClient
rest.get().uri("/{id}", 7)
    .retrieve().body(Notification.class);
```

- исто **синхрон** (чека одговор) како RestTemplate
- „fluent“ стил: метод → адреса → испрати → тело
- грешките по статус ги решаваме со `onStatus`
- ист пакет `spring-web` — нема нова зависност

За нов код во Spring MVC: **`RestClient`** — Spring го препорачува наместо `RestTemplate`

WebClient и кој клиент да го изберам?

WebClient — неблокирачки (spring-webflux)

```
public Mono<Notification> get(Long id) {
    return web.get()
            .uri("/{id}", id)
            .retrieve()
            .bodyToMono(Notification.class);
}               // враќа ВЕДНАШ – одговорот ќе дојде подоцна

public Flux<Notification> all() {
    return web.get().retrieve()
            .bodyToFlux(Notification.class);
}
```

- `Mono<T>` = 0 или 1 вредност „во иднина“
- `Flux<T>` = 0 до N вредности (поток)
- нишката не чека → многу паралелни повици со малку нишки

	RestTemplate	RestClient	WebClient
Од	Spring 3 (2009)	Spring 6.1 (2023)	Spring 5 (2017)
Стил	посебни методи	fluent ланец	fluent, реактивен
Чека одговор?	да	да	**не** (`Mono`/`Flux`)
Зависност	`spring-web`	`spring-web`	`spring-webflux`
Денес	стар код, одржување	**препорачан** за MVC	реактивни апликации

Spring MVC апликација → **`RestClient`** · реактивна (WebFlux) или многу паралелни повици → **`WebClient`**

Ресурси

Книга

- Глава 3.6: MVC шаблон и слоевита архитектура (стр. 136–141)
- Глава 4: Spring MVC (стр. 143–185)
  - 4.1 DispatcherServlet и WebApplicationContext
  - 4.2 Анотирани контролери
  - 4.3 Spring REST и RestTemplate
  - 4.4 Процесирање на барање

Онлајн

- Spring Web MVC — референтна документација: `docs.spring.io/spring-framework/reference/web/webmvc.html`
- Водич: Serving Web Content with Spring MVC — `spring.io/guides/gs/serving-web-content`
- Водич: Building a RESTful Web Service — `spring.io/guides/gs/rest-service`
- Thymeleaf — `thymeleaf.org/documentation.html`